

Project :House Price Prediction

Group :9

1. Introduction:

Problem statement:

- In densely populated areas like California, housing is an issue because the cost of houses is rising faster than people can afford to pay for them, thus making it hard for them to get the appropriate houses. To solve this problem, we have developed an ML project using various features that give accurate predictions of housing prices so that buyers make the right choices when purchasing a house.

Objective:

- The goal of the project is to process the housing dataset using machine learning approaches and conduct efficient analysis.
- To employ different regression models to forecast the housing prices and the factors that affect them, including location, income, and age of the houses. By applying feature engineering and scaling, we were able to establish relationships between these factors and housing prices.
- We applied and assessed various regression algorithms and optimized their parameters for the best results. The model that was chosen was based on evaluation metrics, and the results were used to gain insight into the housing market.

2. Data Exploration and Preprocessing:

Loading and Exploratory Analysis of the California Housing

Loading the Dataset:

The California Housing dataset was pre-processed by using the `fetch_california_housing` function from the `sklearn.datasets` module. This function returns the data in the format of pandas Data Frame.

Understanding the Dataset:

To understand the structure of the dataset, we went through the data types and non-null values of each feature in the given data set by using the command `df.info()`.

The basic summary of the data was done using `df.describe()` to determine the measures of central tendency, variability and data distribution.

We also looked for the presence of missing values using the code `df.isnull().sum()` in order to determine which columns might need imputation or some other form of treatment.

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
count	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000
mean	3.870671	28.639486	5.429000	1.096675	1425.476744	3.070655	35.631861	-119.569704	2.068558
std	1.899822	12.585558	2.474173	0.473911	1132.462122	10.386050	2.135952	2.003532	1.153956
min	0.499900	1.000000	0.846154	0.333333	3.000000	0.692308	32.540000	-124.350000	0.149990
25%	2.563400	18.000000	4.440716	1.006079	787.000000	2.429741	33.930000	-121.800000	1.196000
50%	3.534800	29.000000	5.229129	1.048780	1166.000000	2.818116	34.260000	-118.490000	1.797000
75%	4.743250	37.000000	6.052381	1.099526	1725.000000	3.282261	37.710000	-118.010000	2.647250
max	15.000100	52.000000	141.909091	34.066667	35682.000000	1243.333333	41.950000	-114.310000	5.000010

Data Cleaning and preprocessing:

- **Handling Missing Values:**

We checked that no columns had missing values as shown by `df.isnull().sum()`. If there were cases of missing data, then proper methods of imputation such as mean or median imputation would be applied but as it has been observed the current dataset had no missing values.

- **Outlier Detection:**

We used two methods to detect outliers:

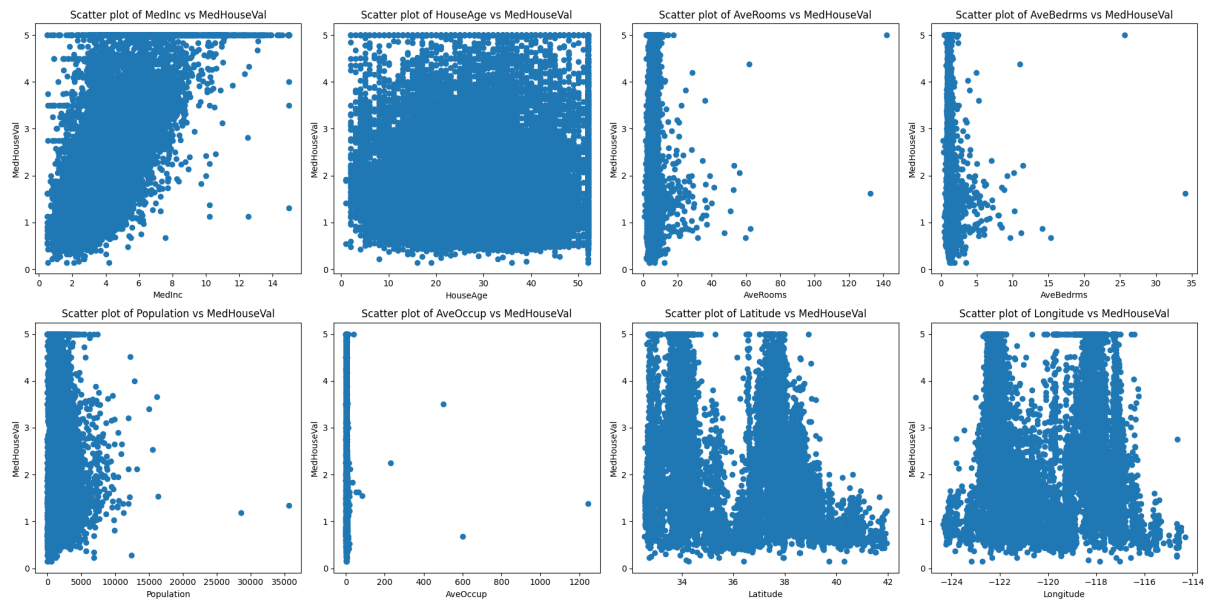
Z-Score Method: We calculated the Z-scores for each feature and marked values with Z-score above 3 as outliers.

IQR Method: We also calculated the IQR for each feature, and then any value outside the range of $Q1 - 1.5 * IQR$ to $Q3 + 1.5 * IQR$ was considered an outlier.

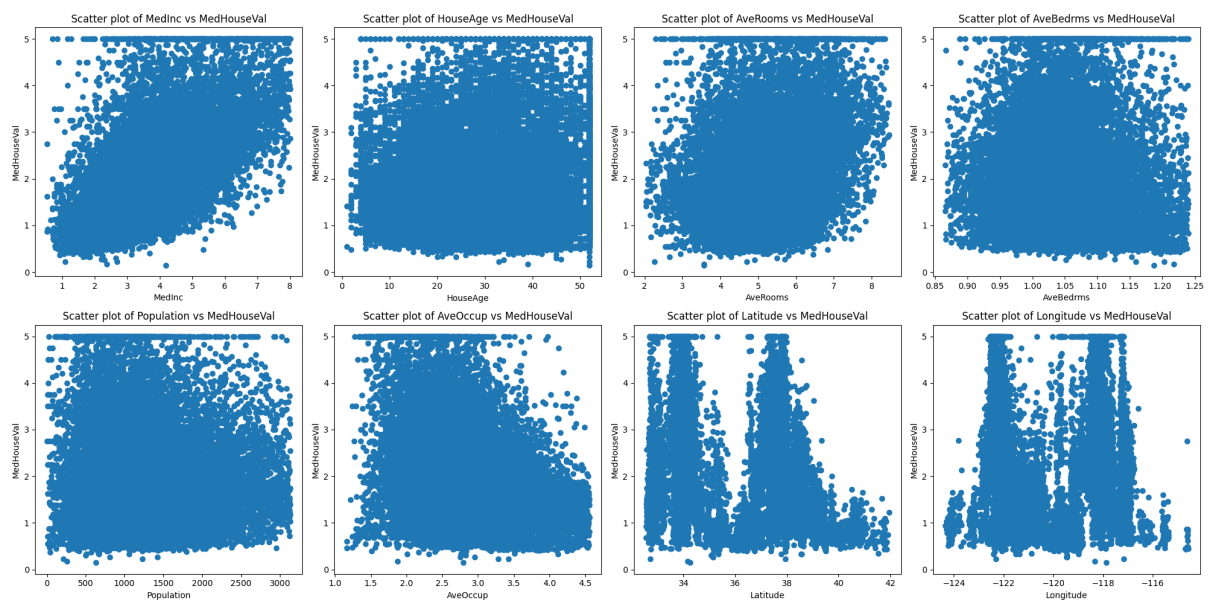
- **Outliers:**

Finally, we identified the outliers and deleted **3798** rows that contained the outlier values through Boolean indexing and plotted the scatter plot to view and compare the results before and after removing outliers.

Before removing :



After removal:



Exploratory Data Analysis:

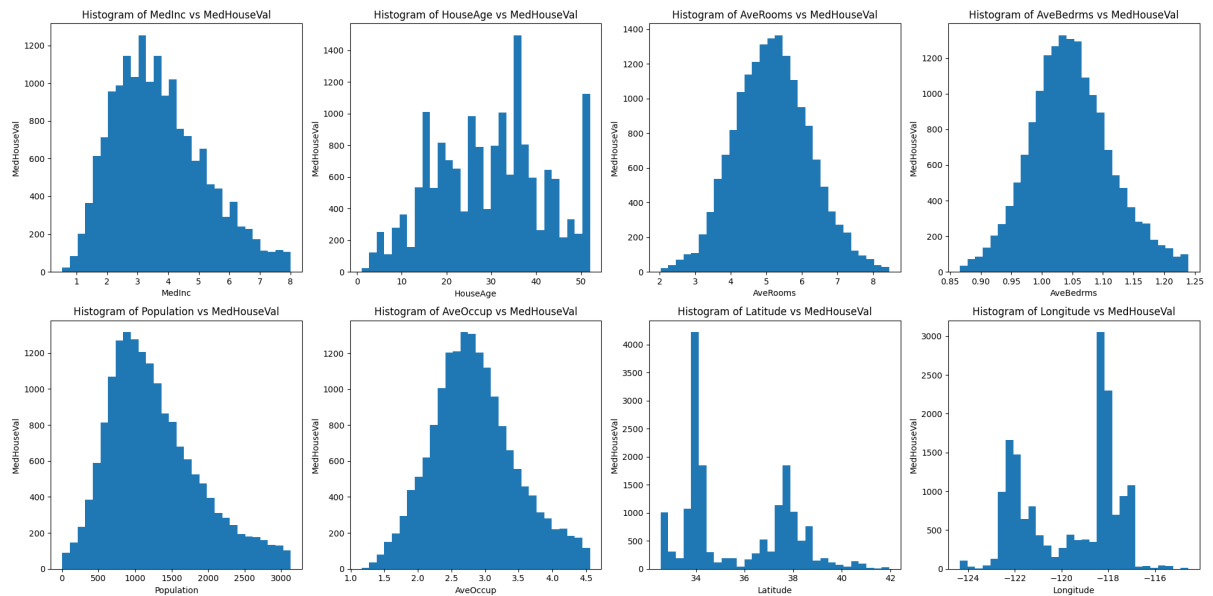
• Scatter Plots:

In order to visualize correlations between single variables and the target variable MedHouseVal, we built scatter plots of the features Latitude and Longitude, Households as well as the other attributes present in the column. These plots assisted in distinguishing trends, and potential relationships between data points, as well as outliers as shown above.

• Histograms:

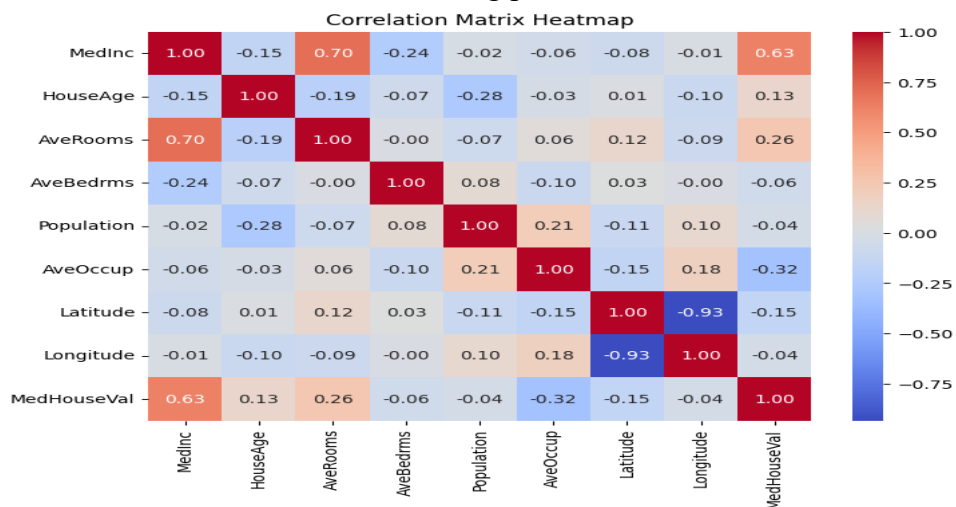
We plotted histograms of the features to get the general idea of the distribution and the existence of skewed distributions and outliers. This was also useful in determining the overall features of the data after preprocessing.

CSCE 5215: Machine Learning



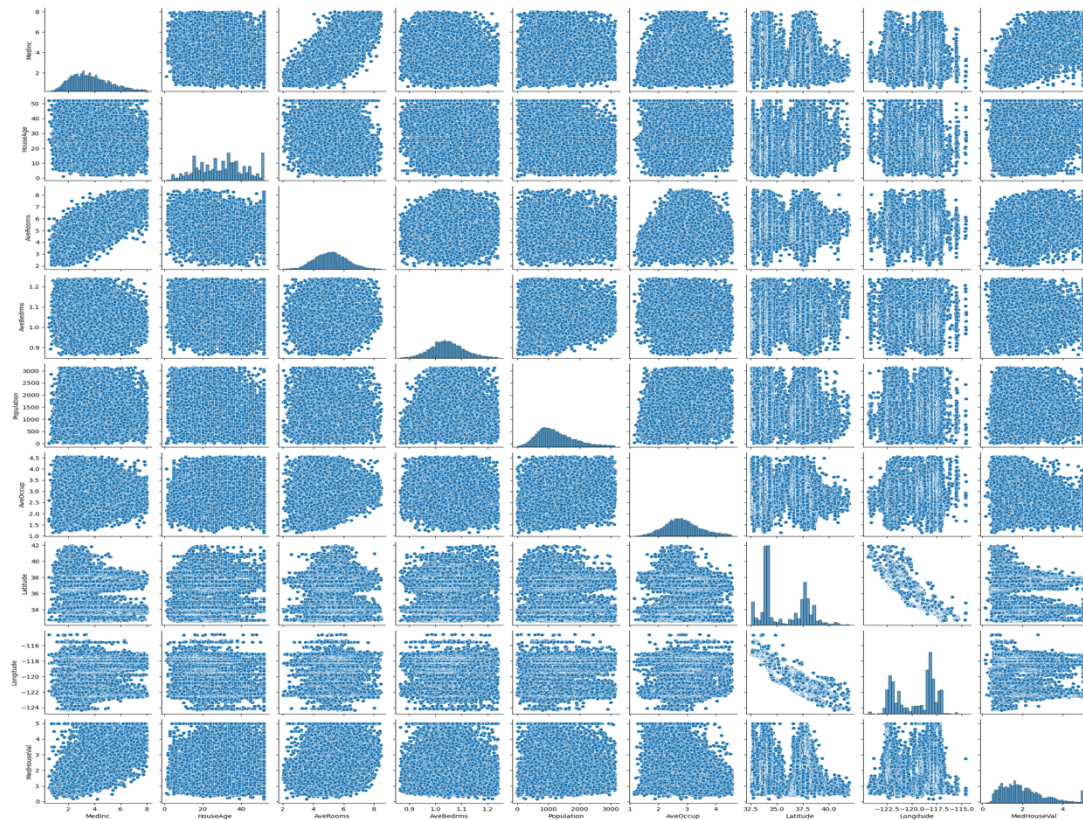
- **Correlation Heatmap:**

To look into the relationships between all features and the target variable, a correlation matrix was computed. To represent this matrix, we created a heatmap in which colors represent the level of correlation. This step assisted us in determining which characteristics constituted strong predictors of the 'MedHouseVal' variable.



- **Pair plot:**

The pairwise relationship between the features was analysed using a pair plot. This made it possible to see how the features affect each other and the target variable in question.



3. Feature Engineering:

New Features:

- We selected (Polynomial degree=2) to catch non-linear relationship between features.
- This enables the model to identify complex relations between the columns.

Feature Scaling:

- All features are scaled with **StandardScaler**, so that they contribute to the model equally.
- Scaling eliminates the mean and normalizes features to a unit variance, ensuring features that range over larger values will not dominate the model.

Feature Selection:

- Used Recursive Feature Elimination and Linear Regression to identify the top 10 most important features.
- RFE iteratively removes the least significant features to reduce dimensionality and make the model more efficient.

```
Selected Features after RFE: ['MedInc' 'AveOccup' 'Latitude' 'Longitude' 'MedInc Latitude'
'MedInc Longitude' 'HouseAge Latitude' 'HouseAge Longitude'
'AveRooms Latitude' 'AveRooms Longitude']
```

4. Model Building and Evaluation:

We have evaluated various regression models to compare the performance and improve the accuracy in predicting house prices.

CSCE 5215: Machine Learning

- **Linear Regression:** The linear regression model considers linear relationships between features and the target. The key drawback is that the housing prices generally involves a lot of non-linear data, hence this model does not perform well.
 - Train R^2 : 0.645434
 - Test R^2 : 0.630569
 - Test RMSE: 0.649139
- **Lasso Regression:** The Lasso Regression showed slight improvement through L1 regularization, but still this model is not highly efficient.
 - Train R^2 : 0.000000
 - Test R^2 : -0.000041
 - Test RMSE: 1.068022
- **Decision Tree Regressor:** The Decision Tree Regressor is a non-linear model. It can easily identify complex patterns, but fails due to overfitting because of large dataset.
 - Train R^2 : 1.000000
 - Test R^2 : 0.588827
 - Test RMSE: 0.684831
- **Decision Tree Regressor with Grid Search:** The usage of Grid Search increases the efficiency of Decision Tree performance. But due to overfitting, the prediction accuracy is not optimal.
 - Train R^2 : 0.857657
 - Test R^2 : 0.712356
 - Test RMSE: 0.572795
- **Random Forest Regressor:** Random Forest is an ensemble method. It helps to reduce overfitting and performs better than decision tree model.
 - Train R^2 : 0.973476
 - Test R^2 : 0.791020
 - Test RMSE: 0.488229
- **Random Forest Regressor with Random Search:** The hyperparameter optimization with Random Search improves the performance slightly, but not as efficient as XGBoost model.
 - Train R^2 : 0.995402
 - Test R^2 : 0.796621
 - Test RMSE: 0.481642
- **XGBoost Regressor:** It is a gradient boosting model. It can handle non-linear relationships and has high efficiency.
 - Train R^2 : 0.944664
 - Test R^2 : 0.810878
 - Test RMSE: 0.464454
- **XGBoost Regressor with Random Search:** By applying Random Search for hyperparameter tuning, the XGBoost offers superior performance in comparison to all other models.
 - Train R^2 : 0.953837
 - Test R^2 : 0.823585

CSCE 5215: Machine Learning

- Test RMSE: 0.448580
- **Voting Regressor:** This ensemble model combines the predictions of multiple models like Linear Regression, Random Forest and XGBoost.
 - Train R^2 : 0.960222
 - Test R^2 : 0.804617
 - Test RMSE: 0.472079
- **Stacking Regressor:** The Stacking Regressor model combines multiple base models, it involves a trained meta-model, to make final prediction.
 - Train R^2 : 0.967748
 - Test R^2 : 0.823330
 - Test RMSE: 0.448904

5. Results and Analysis:

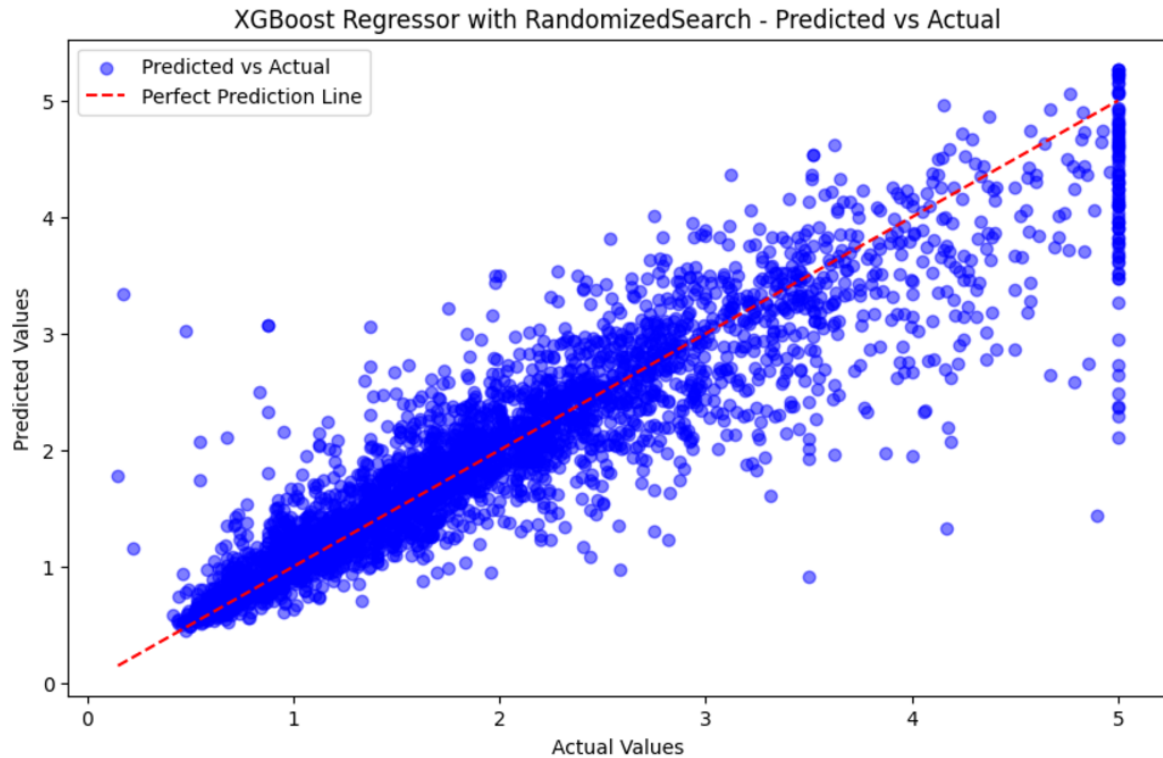
Performance Analysis:

	Model	Train MAE	Train MSE	Train RMSE	Train R^2	Test MAE	Test MSE	Test RMSE	Test R^2
0	Linear Regression	4.689194e-01	4.080057e-01	6.387533e-01	0.645434	0.479537	0.421382	0.649139	0.630569
1	Lasso Regression	8.474663e-01	1.150720e+00	1.072716e+00	0.000000	0.845521	1.140672	1.068022	-0.000041
2	Decision Tree Regressor	1.048173e-17	2.195672e-32	1.481780e-16	1.000000	0.434265	0.468994	0.684831	0.588827
3	Decision Tree Regressor with Grid Search	2.670463e-01	1.637967e-01	4.047181e-01	0.857657	0.374110	0.328094	0.572795	0.712356
4	Random Forest Regressor	1.141241e-01	3.052202e-02	1.747055e-01	0.973476	0.318411	0.238368	0.488229	0.791020
5	Random Forest Regressor With Random Search	4.256674e-02	5.291084e-03	7.273983e-02	0.995402	0.316070	0.231979	0.481642	0.796621
6	XGBoost Regressor	1.787219e-01	6.367581e-02	2.523407e-01	0.944664	0.310606	0.215718	0.464454	0.810878
7	XGBoost Regressor With Random Search	1.608168e-01	5.312107e-02	2.304801e-01	0.953837	0.292663	0.201224	0.448580	0.823585
8	Voting Regressor	1.433889e-01	4.577391e-02	2.139484e-01	0.960222	0.306659	0.222858	0.472079	0.804617
9	Stacking Regressor	1.337571e-01	3.711255e-02	1.926462e-01	0.967748	0.291670	0.201514	0.448904	0.823330

Model Selection:

After evaluating all the models, the **XGBoost Regressor with Randomized Search** is selected as the best model. This model makes predictions with high accuracy, and the outputs clearly project that feature engineering and hyperparameter optimization has high impact on the performance improvement.

- **High Performance:** XGBoost surpassed all other models in each of the training and test evaluation metrics. It has the highest Test R^2 score of 0.8235, which captures greater variance in the target variable, and least Test RMSE of 0.4485, which indicates higher accuracy while making predictions.
- **Hyperparameter Tuning:** To improve the performance of the model, the Random Search allows to alter the hyperparameters to increase the accuracy.
- **Computational Efficiency:** XGBoost model is highly efficient to handle large datasets. This model can be trained quickly while maintaining high performance.



6. Conclusion:

This project shows the procedure of predicting median house value using various machine learning regression models, analysing the model performances, and choosing the optimised and best model. By using various evaluation metrics, XGBoost Regressor with Random Search is proved to be the best model for this dataset. This model outperformed all other models in each of the evaluation metrics and offers a superior predictive accuracy.

7. References:

- Scikit-learn Documentation: [scikit-learn: machine learning in Python — scikit-learn 1.5.2 documentation](https://scikit-learn.org/stable/index.html)
- Scikit California Housing Dataset: [fetch california housing — scikit-learn 1.5.2 documentation](https://scikit-learn.org/stable/datasets/real_world.html#california-housing-dataset)